School of Future Tech

Case Study Report

on

REGULAR EXPRESSION REGULAR EXPRESSION DEBUGGER & TESTER

by

Name :Sasanka Sekhar Kundu

Roll no: 150096725118

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

Regular Expressions (Regex) are a powerful tool used in programming and data processing to search, match, and manipulate text based on specific patterns. They are widely used in form validation, log analysis, data extraction, and text processing. However, regex syntax can be complex and difficult to understand, especially for beginners.

This case study focuses on the design and implementation of a Regular Expression Debugger G Tester, a web-based tool that allows users to write, test, and debug regular expressions interactively. The system highlights matches in real time, displays match details and capture groups, supports regex flags, provides replacement previews, and includes a built-in cheat sheet and theme toggle to improve usability and learning.

The case study demonstrates how frontend web technologies (HTML, CSS, and JavaScript) can be combined to create an effective developer utility that simplifies regex learning and debugging.

---

2. Problem Statement / Case Background (Abstract)

Background

Writing correct regular expressions is often error-prone due to complex syntax, special characters, and flags. Traditional development environments do not always provide immediate visual feedback, making debugging difficult and time-consuming.

A lightweight, browser-based regex debugger helps developers and students quickly test patterns, understand matches, and refine expressions without switching tools or writing additional code.

Abstract

This case study presents the design and implementation of a Regular Expression Debugger G Tester using client-side web technologies. The system accepts user-defined text, regex patterns, and optional replacement strings, dynamically evaluates the expression using JavaScript's `RegExp` engine, and visually highlights matches. It also displays match positions,

capture groups, and replacement results. The application improves learning and debugging efficiency through a structured UI, interactive features, and an integrated regex cheat sheet.

---

3. Case Study Design

The Regex Debugger system is designed as a modular, interactive web application with the following components:

User Interface (HTML G CSS)

- **Text area for input text.**
- **Input field for regex pattern.**
- **Checkboxes for regex flags (`g`, `i`, `m`) aligned in a single row.**
- **Input field for replacement text.**
- **Buttons for testing regex, clearing input, toggling theme, and opening the cheat sheet.**
- **Results section to display highlighted matches and detailed match information.**
- **Modal popup for the regex cheat sheet.**
- **Black G gold theme (default) with optional light mode.**

Regex Processing Logic (JavaScript)

- **Dynamic construction of the regular expression using user input.**
- **Safe handling of invalid regex patterns.**
- **Matching and highlighting of all occurrences.**
- **Display of match indices and capture groups.**
- **Optional replacement preview.**
- **Theme switching and modal control.**

---

4. Methods G Algorithms / Technology Applied

Core Techniques Used

Text Processing

- **JavaScript `RegExp` object creation.**

- **Dynamic flag handling using checkbox input.**
- **Global matching using `matchAll()`.**

Pattern Matching G Debugging

- **Highlighting matched text using HTML `<span>` elements.**
- **Offset handling to preserve correct indices during highlighting.**

User Interaction G UI Control

- **DOM manipulation using `getElementById()` and `innerHTML`.**
- **Event-driven execution through button clicks.**
- **Modal visibility control for the cheat sheet.**
- **Theme switching using CSS class toggling.**

Technology Stack Used

- **Frontend: HTML5, CSS3**
- **Programming Language: JavaScript (ES6)**
- **Environment: Web Browser (Chrome, Safari, Brave)**
- **Tools: VS Code, Live Server**

---

5. Implementation Details and Snapshots

The implementation is organized into three main files:

index.html

- **Defines the structure of the application.**
- **Contains form elements, buttons, result containers, and modal layout.**

style.css

- **Implements a black G gold default theme.**
- **Defines layout, typography, highlights, and modal design.**
- **Includes light-mode overrides using a `.light` class.**
- **Handles responsive layout and scrolling cheat-sheet modal.**

script.js

- **Handles regex evaluation and debugging logic.**
- **Dynamically highlights matches.**
- **Displays match details and capture groups.**
- **Implements replacement preview functionality.**
- **Controls theme toggling and cheat-sheet popup behavior.**

Snapshots That Can Be Included

- **Main interface showing text input, regex pattern, and flags.**
- **Highlighted output displaying matched text.**
- **Match details section with indices and capture groups.**
- **Cheat sheet popup listing common regex patterns.**
- **Light mode and dark mode comparison view.**

---

6. Results and Conclusion

Findings

- **Visual highlighting significantly improves understanding of regex behavior.**
- **Displaying capture groups and indices helps users debug complex patterns.**
- **Integrated cheat sheet reduces the learning curve for beginners.**
- **Theme switching improves accessibility and user comfort.**
- **Client-side execution ensures fast response without server dependency.**

Conclusion

This case study demonstrates the successful development of a Regular Expression Debugger G Tester using standard web technologies. The application provides an intuitive and interactive environment for testing and understanding regex patterns. By combining real-time feedback, detailed match analysis, and supportive UI features, the system serves as an effective learning and debugging tool for students, developers, and educators. The project can be further extended with live matching, regex explanations, and pattern sharing features.

---

7. References

1. **MDN Web Docs – Regular Expressions in JavaScript**
2. **MDN Web Docs –** `RegExp`, `matchAll()`
3. **HTML5, CSS3, and JavaScript official documentation**
4. **Tutorials on Regular Expressions and text pattern matching**
5. **Web accessibility and UI/UX design references**

○